

数据结构

夏天

`xiat@ruc.edu.cn`

中国人民大学信息资源管理学院

栈和队列

栈和队列：两种特殊的线性表

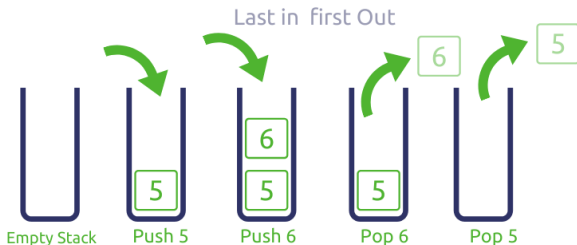
栈/Stack

- 限制仅在表的一端进行插入和删除。
- 通常称插入、删除的一端为栈顶 (Top)，另一端为栈底 (Bottom)
- LIFO: Last In First Out

队/Queue

- 限制仅在表的一端进行插入、在另一端进行删除。
- 允许插入的一端称队尾 (rear)，允许删除的一段称为队头 (front)。
- FIFO: First In First Out

Stack



- 操作系统中的栈

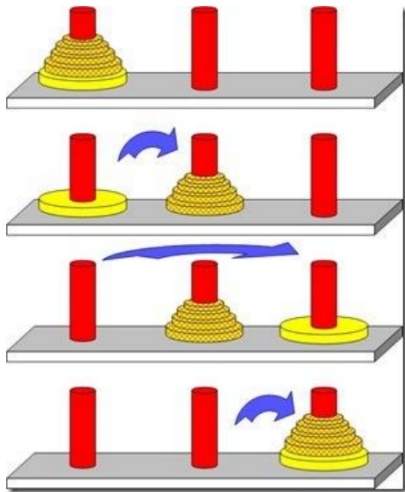
- * 由编译器自动分配释放，存放函数的参数值，局部变量的值等。栈使用的是一级缓存，被调用时处于存储空间中，调用完毕立即释放。

- 数据结构中的栈

- * 一种后进先出的数据结构

为什么设计栈、研究栈?

- 栈的一个重要应用是在程序设计中实现递归，从而使许多实际问题大大简化。



- 上帝创造世界的时候做了三根金剛石柱子，在一根柱子上从下往上按大小顺序摞着 64 片黄金圆盘。上帝命令婆罗门把圆盘从下面开始按大小顺序重新摆放在另一根柱子上，并且规定一次只能移动一个圆盘，在小圆盘上不能放大圆盘。
- 有预言说，这件事完成时宇宙会在一瞬间闪电式毁灭。也有人相信婆罗门至今还在一刻不停地搬动着圆盘。
- 18,446,744,073,709,551,615 次搬动才能挪完 64 片金盘！

举例：计算 n 的阶乘

```
1  int factorial
   ↪  (int n) {
2      int f ;
3      if (n==1)
   ↪  f=1;
4      else f=n*fact(n-1);
   ↪  return f;
5
6 }
```

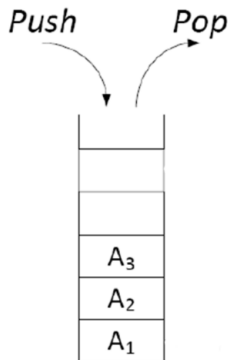
- 1 将调用函数的现场 (各寄存器的值, 中断时的程序地址等) 入栈, 转入被调函数;
- 2 执行被调函数, 如又调用其它函数, 则执行上述步骤;
- 3 被调函数执行完, 取栈顶的值, 恢复调用函数时的现场, 根据现场中的指令地址, 恢复调用函数在中断处继续执行。

$$\begin{aligned} \text{factorial}(4) &= 4 \cdot \text{factorial}(3) \leftarrow \text{return } 3 \cdot 2 \cdot 1 \\ &\quad \swarrow \text{return } 2 \cdot 1 \\ &\quad \swarrow \text{return } 1 \end{aligned}$$

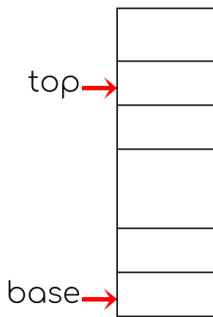
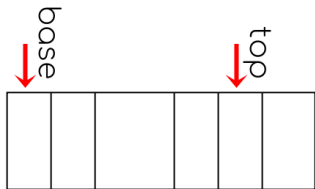
栈的存储表示方法

请考虑其常用操作。试想选用顺序存储还是链式存储？

- 特点：后进先出
 - * 1、经常性的在栈顶插入新元素，以及取栈顶元素；
 - * 2、无须访问非栈顶元素。



1. 顺序栈



- ① 顺序栈中元素用地址连续的存储单元依次存放;
- ② 栈底位置固定不变;
- ③ 栈顶位置 **top** 随着进栈和出栈的操作而变化。

顺序栈类型定义:

```
1      class stack<Elem>{  
2          Object[] data;  
3          int top;  
4          int maxSize;  
5      } SeqStack;
```


- 动态分配

- * 先为栈分配一个初始容量，在栈的空间不够使用时再逐段扩大。

- 指针 base

- * 始终指向栈底位置，如果 base 为 NULL 表示栈不存在；

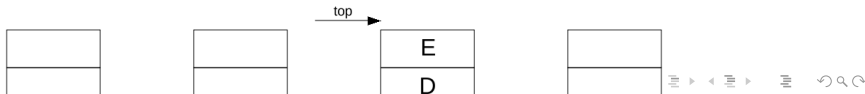
- 指针 top

- * 初值指向栈底，即 $top = base$ ，表示栈空；(不唯一)

- * 每当插入新的栈顶元素，指针 $top++$ ；

- * 每当删除栈顶元素，指针 $top--$ ；

- * 非空栈中的栈顶指针始终在栈顶元素的下一个位置上。



对于顺序栈，入栈需要判栈是否满，是则需要中止、或重新分配空间，否则出现空间溢出。

```
1 public Boolean push
   ↪ (Elem e) {
2     if ( top==maxSize) {
3         print(" 栈已满");
4         return false;
5     }
6     data[top++] = e;
7     return true;
8 }
```

对于顺序栈，入栈需要判栈是否满，是则需要中止、或重新分配空间，否则出现空间溢出。

```
1 public Boolean push  
   ↪ (Elem e) {  
2     if ( top==maxSize) {  
3         print(" 栈已满");  
4         return false;  
5     }  
6     data[top++] = e;  
7     return true;  
8 }
```

出栈首先要判断栈是否为空；否则栈空时进行操作将出现下溢错误。

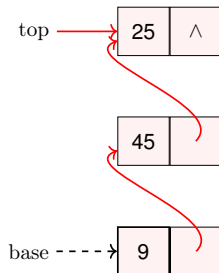
```
1 public Elem pop() {  
2     if(top==base) {  
3         return null;  
4     } else {  
5         top = top-1  
6         return data[top];  
7     }  
8 }
```

2. 链栈

- 链栈有无栈满溢出问题?
- 指针如何指?
 - * 从栈顶依次向后指, 因为操作主要是在栈顶插入、删除, 经常需要根据栈顶元素找次顶元素。
- 是否要加头结点?
 - * No. 因为头部插入不会出现处理不一致的问题。

写出链栈的类型定义:

```
1      class Node{  
2          Elem data;  
3          Node next;  
4      }  
  
5  
6      Node top ; // 链栈
```

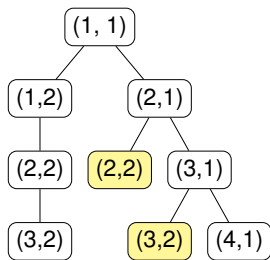


请尝试写出入栈出栈的算法

```
1  public push (Elem e) {  
2      //是否还要判断栈是否满?  
3      top=new Node(e, top);  
4      return OK;  
5  }
```

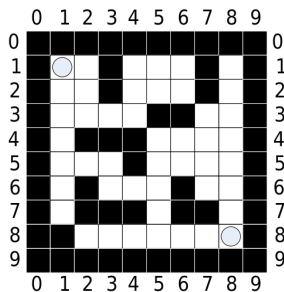
```
6  
7  public pop( ){  
8      //是否还要判断栈是否空?  
9      e=top;  
10     top=top.next;  
11     e.next=null;  
12     return e.data;  
13 }
```

栈的应用 — 迷宫求解



● 处理思路：

- * 对可通行路径的空间的**深度优先搜索**
- * 如果当前位置是出口，找到结果
- * 否则，保存当前位置，进入到下一个可以走的位置，探索路径
- * 如果上一步没有成功，继续探索下一个可以走的位置。



队/Queue

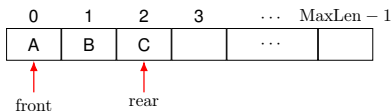
- 限制仅在表的一端进行插入、在另一端进行删除。
- 允许插入的一端称队尾 (rear)，允许删除的一段称为队头 (front)。
- FIFO: First In First Out



- 比如生活中排队购物、操作系统中的作业排队等。

队的存储表示方法

- 队也有两种存储表示方法: 顺序队、链队
- 顺序队: 利用地址连续的存储单元依次存放数据元素。



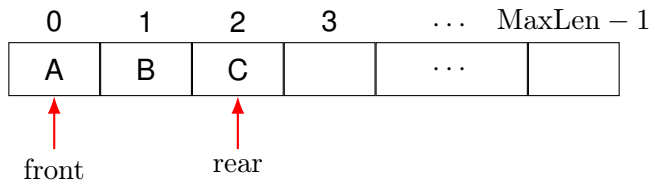
请写出顺序队的类型定义:

```
1      class SeqQueue{  
2          ElemType  data[ ];  
3          int  maxsize;  
4          int  rear; //队尾,  
                    ↪  以此计算队长  
5      }
```

可否??

顺序队的基本操作

- 入队：如有空间，元素 x 入队后队尾指针加 1
 - * $\text{data}[\text{rear}++] = x;$
- 出队：如有元素，队头指针加 1，表明队头元素出队。
 - * $x = \text{sq.data}[\text{sq.front}++];$



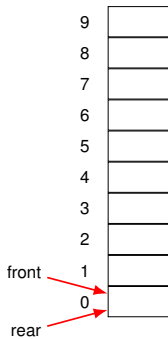
● 设 $\text{maxsize}=10$ 。请你写出如下状态或执行某操作后的顺序队元素。

① 空队；

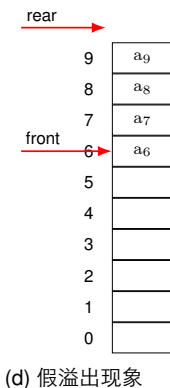
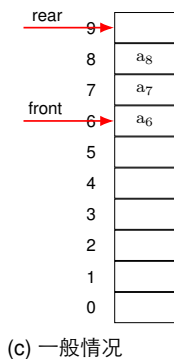
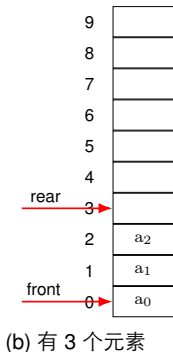
② a_0, a_1, a_2 依次入队；

③ $a_3, a_4, a_5, a_6, a_7, a_8$ 依次入队, $a_0, a_1, a_2, a_3, a_4, a_5$ 依次出队；

④ a_9, a_{10} 入队。



(a) 空队



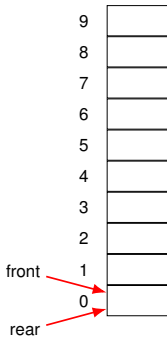
● 设 $\text{maxsize}=10$ 。请你写出如下状态或执行某操作后的顺序队元素。

① 空队；

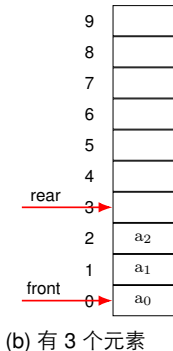
② a_0, a_1, a_2 依次入队；

③ $a_3, a_4, a_5, a_6, a_7, a_8$ 依次入队, $a_0, a_1, a_2, a_3, a_4, a_5$ 依次出队；

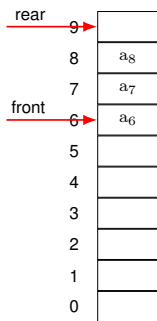
④ a_9, a_{10} 入队。



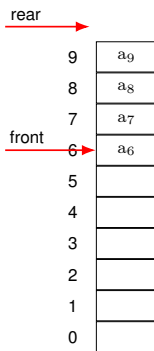
(a) 空队



(b) 有 3 个元素



(c) 一般情况



(d) 假溢出现象

● 入队出队会使整个队列整体后移

● “假溢出”：队尾指针已经移到了最后, 不能再执行入队操作, 但此时队中并未真的“满员”。如何解决这个问题？

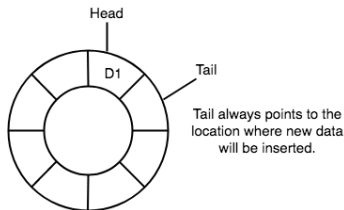
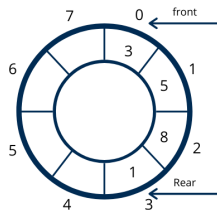
- Linear Queue 入队出队会使整个队列整体后移
- “假溢出”：队尾指针已经移到了最后, 不能再执行入队操作, 但此时队中并未真的“满员”。如何解决这个问题?
 - ① 平移元素;
 - ② 循环队列 (Circular Queue): 将顺序队列的存储区假想为环状空间。

在假溢出时, 将新元素插入到第一个位置上, 这样做, 虽物理上队尾在队首之前, 但逻辑上队首仍然在前。入列和出列仍按“先进先出”的原则进行。

 - 入队时的队尾指针操作:
 $\text{rear} = (\text{rear} + 1) \% \text{maxsize};$
 - 出队时的队头指针操作:
 $\text{front} = (\text{front} + 1) \% \text{maxsize};$

循环队列 (Circular Queue)¹

- head/front points to the first (oldest) used element — the next element to be read
- tail/rear points to the first (oldest) unused element — the next element to be written
- rear: 多数情况下指向尾部元素, tail: 多数情况下指向尾部的下一个元素。
教材中的 rear 等同于 tail



¹延伸阅读: https://en.wikipedia.org/wiki/Circular_buffer

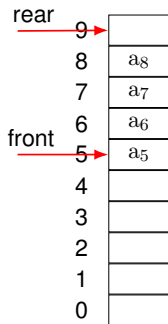
- 设 $\text{maxsize}=10$ 。队列状态如 (a) 所示。请写出分别执行如下操作后的 front 和 rear 值：
 - * 情况 1: $a_9, a_{10}, a_{11}, a_{12}, a_{13}, a_{14}$ 依次入队
 - * 情况 2: a_5, a_6, a_7, a_8 依次出队

- 设 $\text{maxsize}=10$ 。队列状态如 (a) 所示。请写出分别执行如下操作后的 front 和 rear 值:

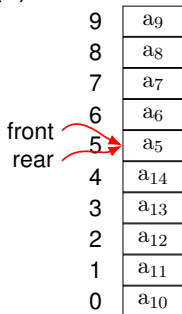
* 情况 1: $a_9, a_{10}, a_{11}, a_{12}, a_{13}, a_{14}$ 依次入队

* 情况 2: a_5, a_6, a_7, a_8 依次出队

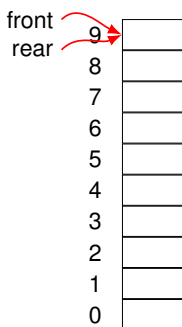
(a) 有 4 个元素



(b) 情况 1: 队满



(c) 情况 2: 队空



当 $\text{front}=\text{rear}$, 队满还是队空? 如何判断队满? 如何判断队空?

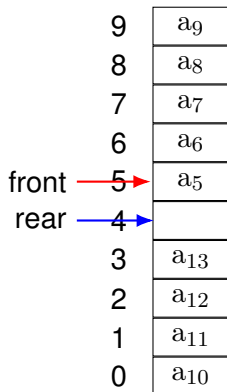
循环队的队满判别

- 方法一

- * 附设一个存储队中元素个数的变量如 num ，当 $num=0$ 时队空，当 $num=maxsize$ 时为队满。

- 方法二

- * 少用一个元素空间，把右图所示情况视为队满，此时的状态是队尾指针加 1 就会从后面赶上队头指针： $(rear+1) \% maxsize == front$ ，从而和空队区别开来。



循环队的基本操作

```
1  class SeqQueue {
2      ElemType data[];
3      int maxsize;
4      int front, rear; /* 队头队尾指针 */
5      int num;         /* 队中元素的个数 */
6  }
7
8  // 入队
9  int InQueue (ElemType x){
10     if (num==maxsize) {
11         printf(" 队满"); return 0;
12     } else {
13         data[rear]=x;
14         rear=(rear+1) % maxsize;
15         num++;
```

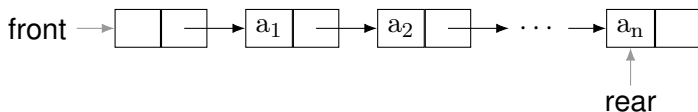
循环队的基本操作

```
1 //出队
2 int OutQueue (ElemType x) {
3     if (num==0) {
4         printf(" 队空");
5         return 1;
6     } else {
7         x=data[front]; /* 读出队头元素 */
8         front=(front+1) % maxsize;
9         num--;
10        return 1;      /* 出队完成 */
11    }
12 }
```

Python:

```
1 class SeqQueue:
2     def __init__(self, max_size=10):
3         self.max_size = max_size + 1 # 实际多放一个空间,
4         ↪ 区分对空、队满
5         self.front = 0
6         self.rear = 0
7         self.elements = [None]*max_size
8
9     def enqueue(self, element) -> bool:
10        if (self.rear + 1) % self.max_size ==
11            ↪ self.front:
12            print('Queue is Full!')
13            return False
14        else:
15            self.elements[self.rear] = element
16            self.rear = (self.rear + 1) % self.max_size
```

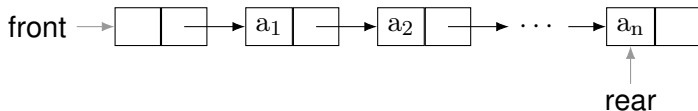
链队



- 带头结点吗?
- 尾指针 rear 指向队尾元素，提高插入操作效率
- 请写出链队的类型定义

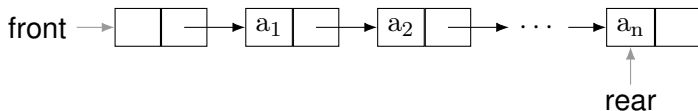
```
1  class QNode{
2      ElemType data;
3      QNode next;
4  }
5  class LQueue{
6      Node head, rear;
```

链队的基本操作 — 入队



```
1 void InQueue(ElemType x) {  
2     rear->next=new QNode(x,null);  
3     rear=rear->next;  
4 }
```

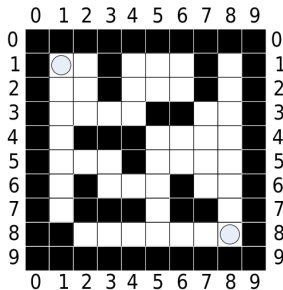
链队的基本操作 — 出队



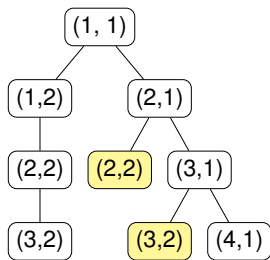
```
1  ElemType OutQueue() {  
2      QNode p;  
3      if (front==rear) {  
4          printf (" 队空");  
5          return null;  
6      } else {  
7          p=front.next;      //把队的第一个结点取出  
8          front.next=p.next; //把队头直接指向原第二个结点  
9          e=p.data;          /* 队头元素放 e 中 */  
10         if (front.next==NULL)  
11             rear=front;    //若只有一个元素，出队后队空，
```

队列应用 — 迷宫寻路

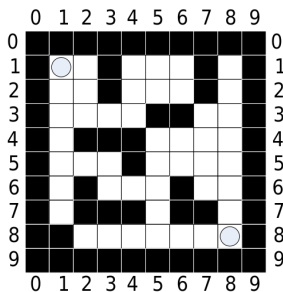
- 从迷宫入口点 (1,1) 出发，向四周搜索，记下所有一步能到达的坐标点；然后依次再从这些点出发，再记下所有一步能到达的坐标点，...，依此类推，直到到达迷宫的出口点 (8,8) 为止，然后从出口点沿搜索路径回溯直至入口。这样就找到了一条迷宫的最短路径，否则迷宫无路径。
- 与基于栈的迷宫求解的异同？



解法图示

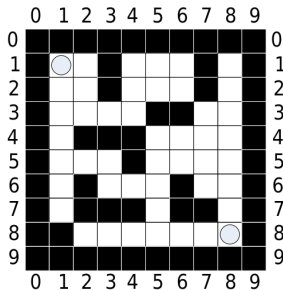


- 上述处理的实质：
 - * 对可通行路径的空间的**广度优先搜索**
- 如何向前摸索？
- 到达出口后，如何回溯？

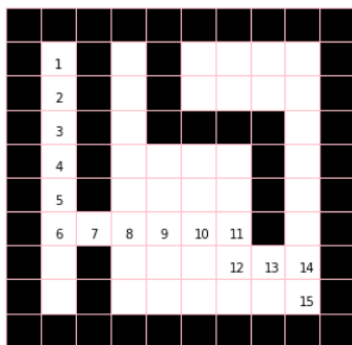
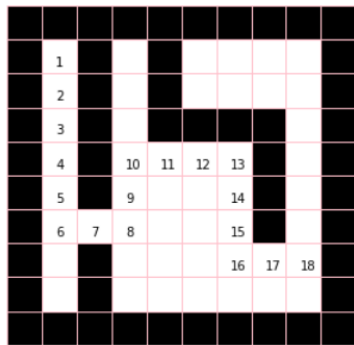


解法图示

- 如何向出口摸索：搜索路径的存储 1
 - * 在搜索过程中必须记下每一个可达到的坐标点，以便从这些点出发继续向四周搜索，使用“FIFO”的队列保存已到达的点即可。
- 如何向入口回溯：搜索路径的存储 2
 - * 为了能够从出口点沿搜索路径回溯直至入口，对于每一点，记下坐标点的同时，还要记下到达该点的前驱点或者从前驱点来的方向（8 个方向之一）。



夏老师实现结果的对比



实现代码见 `ipynb/stack.ipynb` 和 `ipynb/queue.ipynb`